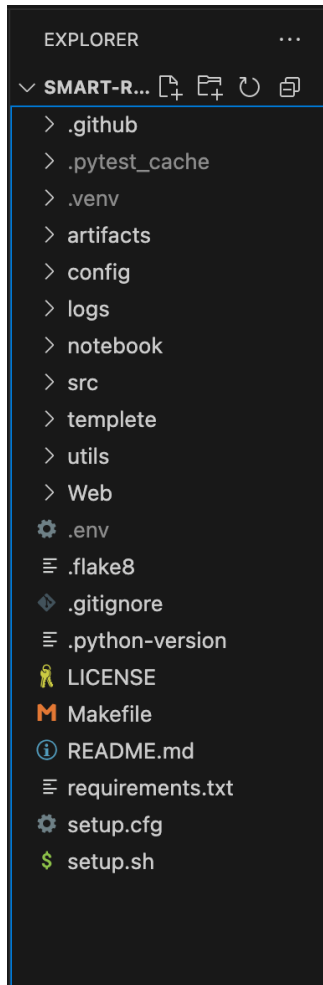


Project Contribution Report

Project Title: Smart Resume Builder

Contributor: Chandan Mohan

Role: Backend Developer & Workflow Architect



Contribution: Project Initialization and GitHub Workflow Setup

One of my primary responsibilities in the Smart Resume Builder project was setting up the foundational structure and workflow for the development team. I created the entire project directory following a modular and scalable architecture. This included structured folders such as `src`, `config`, `utils`, `artifacts`, `logs`, `notebook`, and `Web`, along with essential configuration and environment files like `.env`, `requirements.txt`, `setup.cfg`, `Makefile`, and `.flake8`. This organized structure helped ensure code maintainability and clarity across team members.

In addition to setting up the project architecture, I configured the GitHub workflow to streamline collaboration. I initialized a Git repository and added a `.gitignore` file to exclude environment and cache files. I also established GitHub as our central version control system and created a workflow where all three team members could push and pull code without conflicts. This setup allowed us to work on different modules in parallel and merge updates smoothly.

I further configured the `.github` directory to support actions and automation, laying the groundwork for continuous integration and seamless team contributions. Overall, this initial setup was critical to the project's success, as it ensured that all collaborators could work efficiently from the start.

Contribution: LLM Integration and Named Entity Recognition (NER) Setup

As part of the backend development for the Smart Resume Builder, I worked on designing and implementing the `BaseExtractor` class in the `extractor_fusion.py` file, located within the `src/extractors/` directory. This class was central to our logic, as it served as a unified interface to call and manage various Large Language Model (LLM)-based extractors. It efficiently coordinated multiple LLMs for tasks such as resume parsing and job description understanding, ensuring robust and context-aware data extraction.

The `BaseExtractor` was designed with modularity and extensibility in mind, allowing seamless addition or replacement of individual extractors (e.g., `resume_llm_extractor.py`, `jd_llm_extractor.py`) without altering the core pipeline. This design helped in maintaining a clean architecture while maximizing reusability and performance consistency across different types of input documents.

In parallel, I contributed significantly to the development of the Named Entity Recognition (NER) module, located under `src/ner/`. This module was built to enhance the intelligence of our resume parsing by extracting key resume attributes such as:

- **Action Verbs** – to identify the professional tone and impactfulness of resume statements.
- **Metrics and Impacts** – to capture quantifiable achievements from job experience.
- **Soft and Hard Skills** – to map candidate capabilities to job requirements.

Files like `extract_resume_ner.py`, `ner_predictor.py`, and `ner_service.py` encapsulated the logic to run inference using a trained NER model. These were later integrated with downstream modules like `render_resume.py` to visualize the extracted information and provide feedback.

This dual contribution—developing the LLM integration through the extractor interface and the NER pipeline—played a key role in making our resume builder smart, adaptive, and job-aware.

Contribution: Resume Highlighting and End-to-End Analysis Pipeline

Beyond core extraction and NER components, I contributed to building multiple supporting modules at the `src/` level that played essential roles in enhancing and operationalizing our resume builder.

1. Resume Highlighting (`ner_highlighter.py`)

I developed the logic in `ner_highlighter.py` to dynamically highlight the most impactful elements of a resume based on the NER model's output. This includes keywords like:

- **Action Verbs**
- **Metrics and Results**
- **Soft Skills**
- **Hard Skills**
- **Impactful Phrases**

This functionality helped users instantly see which areas of their resume were strong or needed improvement, especially in relation to a target job description.

2. End-to-End Processing Pipeline (`analyse_pipeline.py`)

I built `analyse_pipeline.py` to serve as a master controller that integrates all components: from extracting information using LLMs and NER to analyzing and enhancing resumes based on job descriptions. This pipeline provides a **production-quality flow** that:

- Fetches and cleans resume and job data
- Uses extractors and predictors to analyze content
- Highlights resumes using insights from NER
- Generates ATS scores and renders the final resume output

3. Job-Description Focused Analysis (`analyze_jd_focus.py`)

I developed this script to analyze job descriptions specifically and extract core expectations from recruiters. It ensured that our resume adjustments were tailored and not generic, increasing the user's chances of selection.

4. ATS Scoring (`ats_score.py`)

I implemented a scoring system to calculate how well a resume matches a job description. This module outputs a score and key gaps between the resume and JD, helping users understand their alignment at a glance.

5. Exception Handling and Logging (`custom_exception.py` & `logger.py`)

To make the system robust and easier to debug, I wrote reusable logging utilities (`logger.py`) and custom error handlers (`custom_exception.py`). These helped us catch, log, and respond to failures gracefully in both development and production environments.

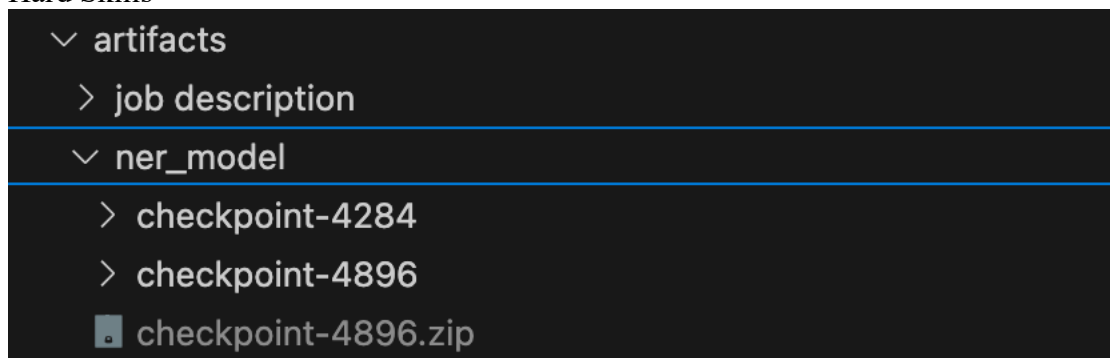
6. Resume Rendering (`render_resume.py`)

Finally, I helped build the rendering module that takes all annotated and processed data to generate a final visual version of the resume—ready for download or display with highlights and ATS feedback integrated

Contribution: Custom NER Model Development and Dataset Creation

To significantly enhance the intelligence and contextual accuracy of our resume parser, I led the development of a custom **Named Entity Recognition (NER)** model tailored to extract resume-relevant labels such as:

- Action Verbs
- Metrics
- Impact Statements
- Soft Skills
- Hard Skills



1. Dataset Creation (20,000 Annotated Samples)

I curated and created a robust dataset of 20,000 annotated samples. This was a combination of:

- Public resume datasets
- Manually converted job/resume examples fetched from platforms like **Waggly**
- Semi-synthetic examples generated using **Large Language Models (LLMs)** to maintain label diversity and coverage

The dataset was annotated to align with our custom label taxonomy, ensuring that it trained the model to detect the exact insights recruiters and ATS systems look for.

2. Model Training and Checkpointing

Using this dataset, I trained multiple versions of the NER model and saved them as checkpoints for comparison and fallback. The models were stored in:

- `checkpoint-4284/`
- `checkpoint-4896/`
- `checkpoint-4896.zip` for packaged deployment

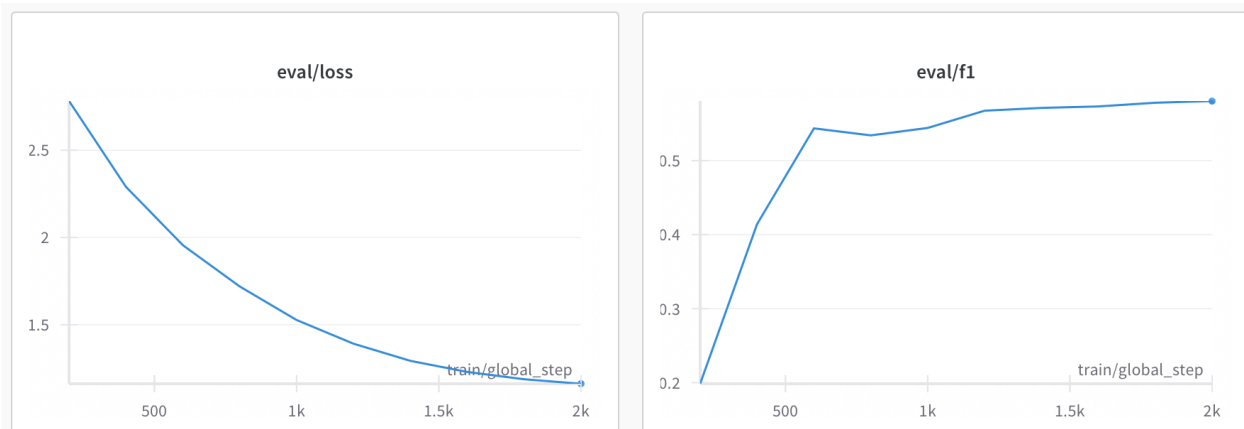
Each checkpoint reflects different configurations of tokenization strategy, label smoothing, or data sampling that we tested to match our requirements. The second model (`checkpoint-4896`) offered the best performance in terms of precision/recall balance and inference time.

3. Performance Improvement and Feedback Loop

The trained model demonstrated **clear improvement** in resume content extraction, especially in isolating impactful statements and relevant skills. I shared evaluation **charts and performance visuals** (available in the notebook) that reflected this improvement.

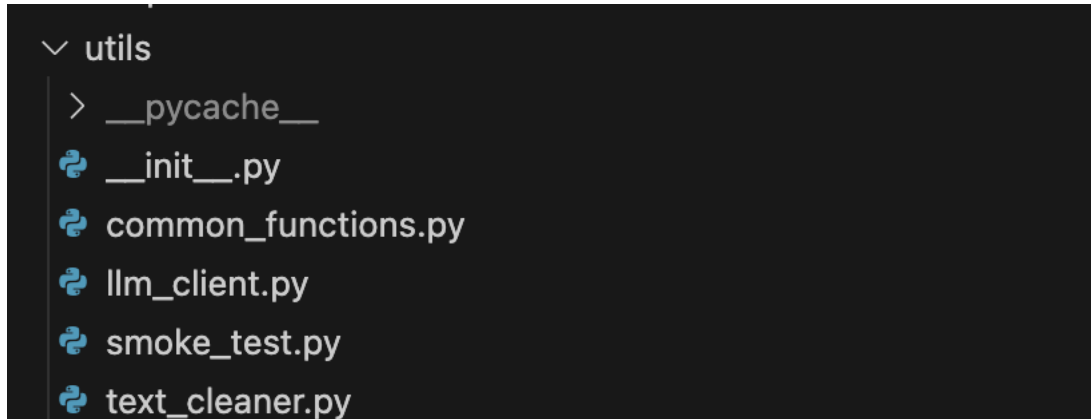
Note: For a clearer understanding of the model's performance, please refer to the evaluation charts showing label-wise F1-score and entity density. These support the case for expanding the dataset further to improve generalization.

This model became a core component of our backend intelligence and powered several downstream modules such as resume rendering, keyword matching, and ATS scoring.



Contribution: Utility Module for Text Processing and LLM Integration

To streamline various recurring operations across the project, I designed and implemented a set of reusable utility scripts under the `utils/` directory. These were essential in reducing redundancy, maintaining consistency, and simplifying the development of higher-level features.



1. Text Extraction and Cleaning

In `text_cleaner.py`, I developed functions to preprocess and sanitize textual data. This included:

- **Regex-based noise removal** (e.g., special characters, redundant whitespace, escape characters)
- **Standardization** of punctuation, date formats, and spacing
- **Lowercasing and token refinement** to prepare input for both NER and LLM models

These preprocessing steps were vital in ensuring consistent input quality across various data sources like PDFs, raw resumes, and job descriptions.

2. PDF to Text Conversion

Within `common_functions.py`, I added helper functions to extract text content from uploaded PDF resumes. This enabled automated ingestion and parsing of resumes submitted by users, ensuring smooth transitions into the processing pipeline.

3. LLM Client Integration

In `llm_client.py`, I wrote an abstraction layer to interface with multiple LLM service providers such as:

- **OpenAI (GPT models)**
- **Groq**
- **Together.ai**

This layer handles API calling, authentication, retries, and formatting—allowing other parts of the system to request LLM services without worrying about provider-specific differences. It also made switching or combining models from different providers easier as we experimented with prompt tuning and cost-performance optimization.